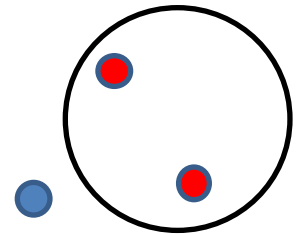
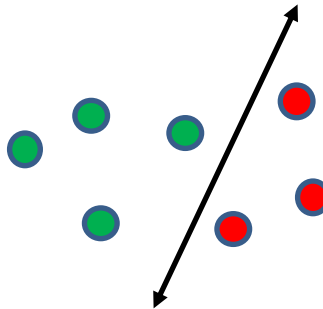
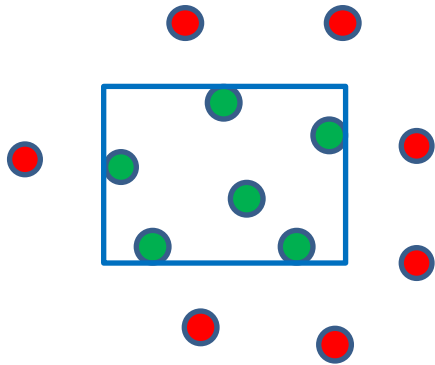


Nearest neighbor classifier

Lee-Ad Gottlieb

Classifiers

- Classifiers we've previously seen
 - Intervals, lines, hyperplanes, rectangles, balls



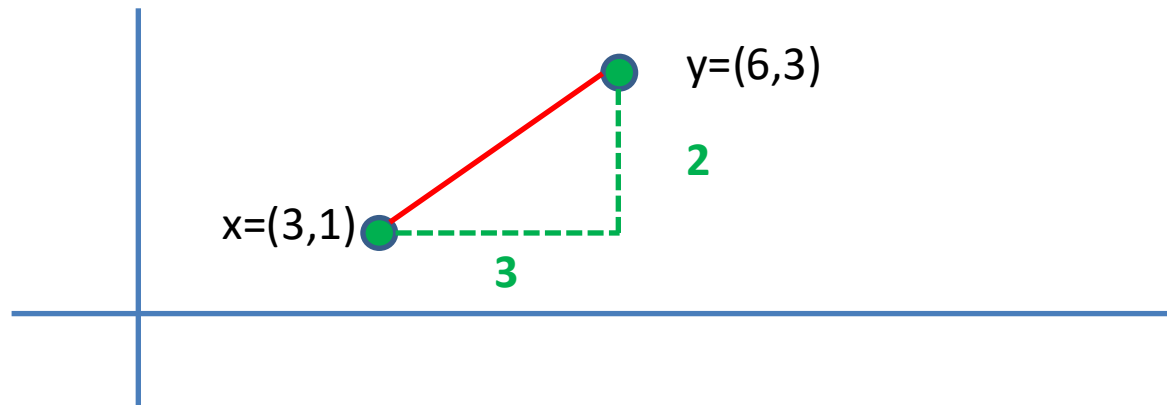
- We've implicitly assumed **Euclidean** space
 - This isn't the only distance measure out there!

Euclidean space

- Euclidean distance:
 - Also known as ℓ_2
 - Distance between vectors x, y :

$$\|x - y\|_2 = \left[\sum_{i=1}^d |x_i - y_i|^2 \right]^{\frac{1}{2}}$$

- Example: $\sqrt{3^2 + 2^2} \sim 3.6$

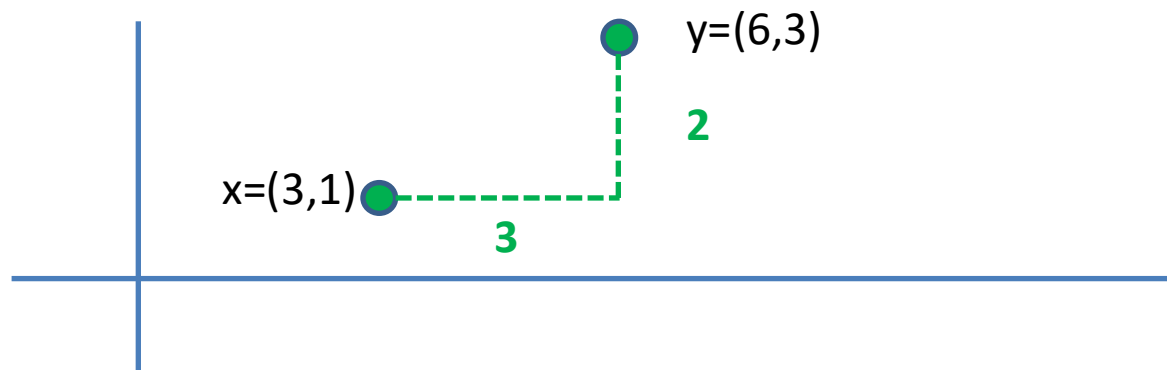


Manhattan norm

- Manhattan distance:
 - Also known as **Taxicab** distance or ℓ_1
 - Distance between vectors x, y :

$$\|x - y\|_1 = \sum_{i=1}^d |x_i - y_i|$$

- Example: $3 + 2 = 5$



By the way...

- Taking a **taxicab** in **Manhattan**
 - But technically, Manhattan is on an angle



ℓ_p space

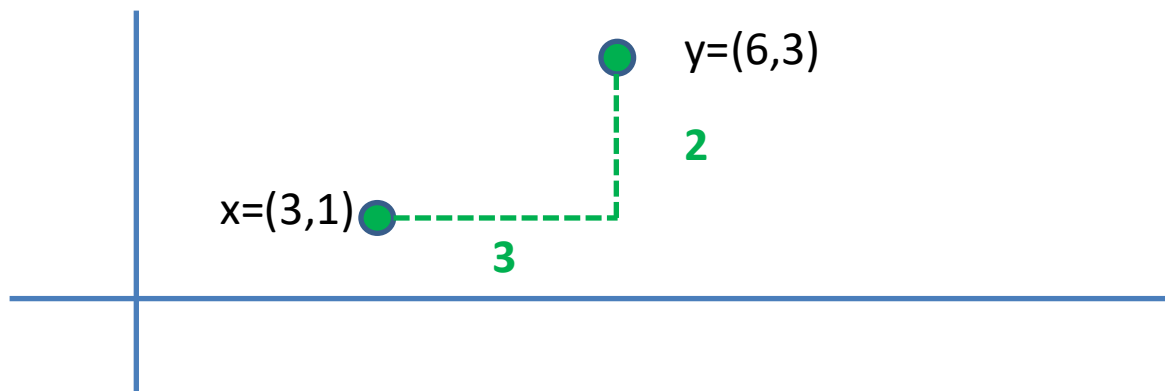
- ℓ_p norms:

- Defined for all $p \geq 0$

- Distance between vectors x, y :

$$\|x - y\|_p = \left[\sum_{i=1}^d |x_i - y_i|^p \right]^{\frac{1}{p}}$$

- Example: $\sqrt[p]{3^p + 2^p}$

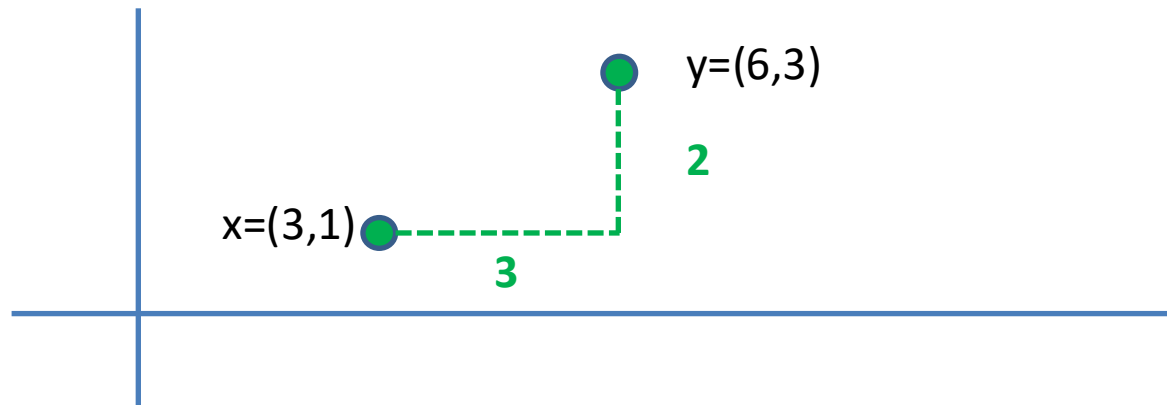


Frechet distance

- Frechet distance:
 - Also known as ℓ_∞
 - Distance between vectors x, y :

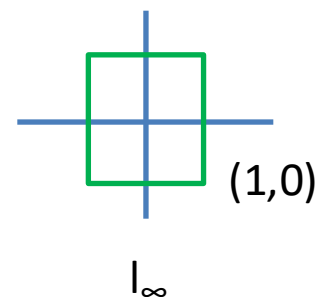
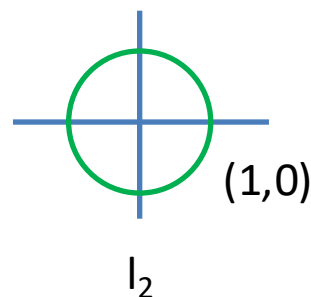
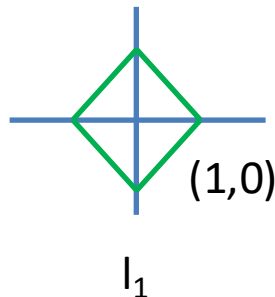
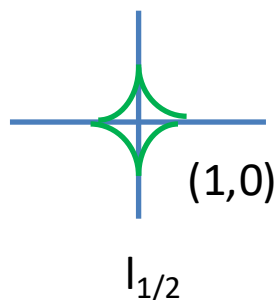
$$\|x - y\|_\infty = \left[\sum_{i=1}^d |x_i - y_i|^\infty \right]^{\frac{1}{\infty}} = \max_i |x_i - y_i|$$

- Example: $\|x - y\|_\infty = \max(3, 2) = 3$



Comparing ℓ_p spaces

- How do the different ℓ_p norms compare?
 - For $x=(3,1)$ and $y=(6,3)$
 - $\|x - y\|_1 = 5$ $\|x - y\|_2 = 3.6$ $\|x - y\|_\infty = 3$
 - For $x=(0,0)$ and $y=(0,1)$
 - $\|x - y\|_p = 1$ for all p
- Explanation for decreasing distances:
 - Unit ball under different p 's

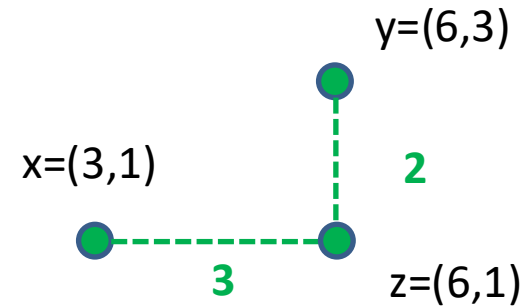


Fractional ℓ_p spaces

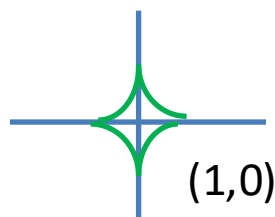
- How does ℓ_p behave when $p < 1$?

– For $x=(3,1)$, $y=(6,3)$ and $z=(6,1)$

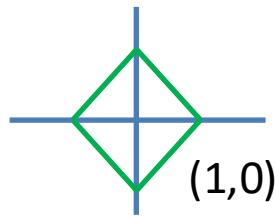
- $\|x - y\|_{\frac{1}{2}} = (3^{1/2} + 2^{1/2})^2 \sim 9.9$
- $\|x - z\|_{\frac{1}{2}} + \|z - y\|_{\frac{1}{2}} = 3 + 2 = 5$
- Violates triangle inequality



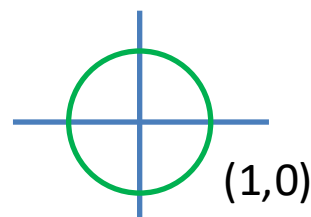
- Concave unit ball



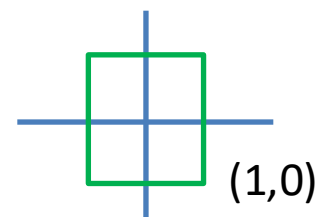
$l_{1/2}$



l_1



l_2



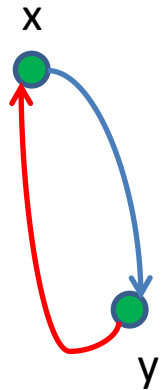
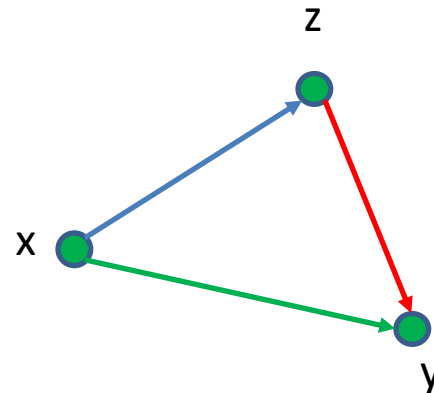
l_∞

What is ℓ_0 ?

- ℓ_0 distance function
 - Distance between vectors x, y : $\left[\sum_{i=1}^d |x_i - y_i|^0 \right]^{\frac{1}{0}}$
- Used to denote the number of non-zero values in vector $x-y$.
 - But really if $x-y=(1,1)$ then $\|x - y\|_0 = \infty$

What is a metric?

- A metric obeys
 - Symmetry: $d(x,y) = d(y,x)$
 - Triangle inequality: $d(x,y) \leq d(x,z) + d(z,y)$
- A semi-metric obeys
 - Symmetry **only**
- A quasi-metric obeys
 - Triangle inequality **only**



Semi-metrics

- Semi-metrics are symmetric, but don't obey triangle inequality
- Examples:
 - ℓ_p when $p < 1$
 - Airline prices: Direct flight more expensive than stopover



Quasi-metrics

- Quasi-metrics obey the triangle inequality, but aren't symmetric
- Examples:
 - Travel distances (uphill, traffic)



Other distance functions

- Consider trying to match a string to its nearest neighbor.
 - Google search
 - autocomplete
- “hello world”
 - Deletion: “helo world”
 - Insertion: “othello world”
 - Substitution: “hallo world”

Levenshtein distance

- Vector distances (ℓ_p) can't handle insertions and deletions
 - Ex: $x=(1,2,3,4,5,7,8,9,10)$ $y=(0,1,2,3,4,5,6,7,8,9)$
 - Only one insertion and one deletion
 - But $\|x - y\|_1 = 10$
- Levenshtein distance on strings:
 - How many insertions, deletions, substitutions to get from x to y .
 - **Insertions** and **deletions** cost 1.
 - **Substitution** either costs 1 or 2 [= insertion + deletion]

DNA

- A good example for Levenshtein distance is for DNA strings

- 4 basic amino acids:

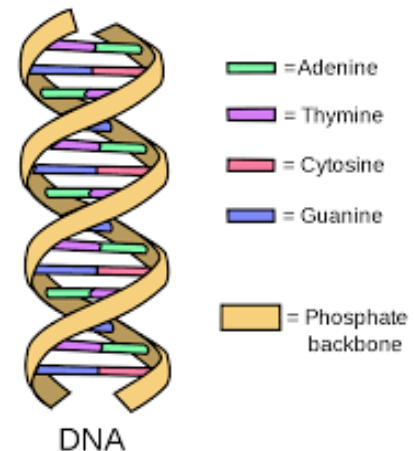
- **A**denine, **C**ytosine, **G**uanine, **T**hymine

- What's the distance from

- GGCTAG to GCGCAA ?

- GGCTAG → G**C**G**C**TAG → G**C**G**C**AG → G**C**G**C**AA**A**

- **I**n**s**ertion, **d**el**e**tion, **s**ub**s**titution = 3



Levenshtein distance

- Also called **edit distance**
- Can be computed as a **dynamic program**
 - Actually computes distances between all prefix of s, t
- Run-time: $O(|s| * |t|)$

DNA – dynamic program

	-	G	C	G	C	A	A
-							
G							
G							
C							
T							
A							
G							

$d(\text{GGCTA}, \text{GCG})$

$d(\text{GG}, \text{GCGC})$

DNA – dynamic program

	-	G	C	G	C	A	A
-	0						
G							
G							
C							
T							
A							
G							

$d(\text{empty string}, \text{empty string}) = 0$

DNA – dynamic program

	-	G	C	G	C	A	A
-	0	1	2	3	4	5	6
G	1						
G	2						
C	3						
T	4						
A	5						
G	6						

 = insertion at cost 1


DNA – dynamic program

	-	G	C	G	C	A	A
-	0	1	2	3	4	5	6
G	1	0	1	2	3	4	5
G	2						
C	3						
T	4						
A	5						
G	6						



= insertion of G to both at cost 0

DNA – dynamic program

	-	G	C	G	C	A	A
-	0	1	2	3	4	5	6
G	1	0	1	2	3	4	5
G	2	1	1	1	2	3	4
C	3						
T	4						
A	5						
G	6						



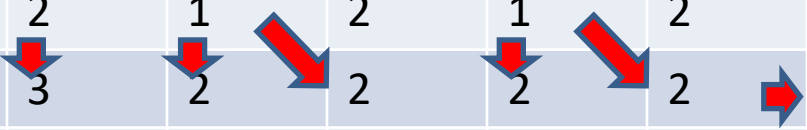
= insertion of G and C = substitution at cost 1

DNA – dynamic program

	-	G	C	G	C	A	A
-	0	1	2	3	4	5	6
G	1	0	1	2	3	4	5
G	2	1	1	1	2	3	4
C	3	2	1	2	1	2	3
T	4						
A	5						
G	6						

DNA – dynamic program

	-	G	C	G	C	A	A
-	0	1	2	3	4	5	6
G	1	0	1	2	3	4	5
G	2	1	1	1	2	3	4
C	3	2	1	2	1	2	3
T	4	3	2	2	2	2	3
A	5						
G	6						



DNA – dynamic program

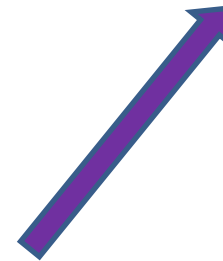
	-	G	C	G	C	A	A
-	0	1	2	3	4	5	6
G	1	0	1	2	3	4	5
G	2	1	1	1	2	3	4
C	3	2	1	2	1	2	3
T	4	3	2	2	2	2	3
A	5	4	3	3	3	2	2
G	6						

The diagram illustrates the alignment between the sequence 'T' and 'A' using the dynamic programming table. Red arrows show matches (T-G) and mismatches (T-C). Green arrows show matches (A-A).

- Red arrow from T[2] (3) to A[2] (4)
- Red arrow from T[3] (2) to A[3] (3)
- Red arrow from T[4] (2) to A[4] (3)
- Red arrow from T[5] (2) to A[5] (3)
- Green arrow from T[6] (2) to A[6] (2)
- Green arrow from T[7] (3) to A[7] (2)

DNA – dynamic program

	-	G	C	G	C	A	A
-	0	1	2	3	4	5	6
G	1	0	1	2	3	4	5
G	2	1	1	1	2	3	4
C	3	2	1	2	1	2	3
T	4	3	2	2	2	2	3
A	5	4	3	3	3	2	2
G	6	5	4	3	4	3	3



Final result

DNA – dynamic program

- Program:

Levenshtein(String s, String t)

```
int[][] a = new array[s.length()+1][t.length()+1]
int count_s = 0, count_t = 0
for (int i=0; i ≤ s.length(); i++) a[i][0] = count_s++
for (int j=0; j ≤ t.length(); j++) a[0][j] = count_t++
```

\\ first column

\\ first row

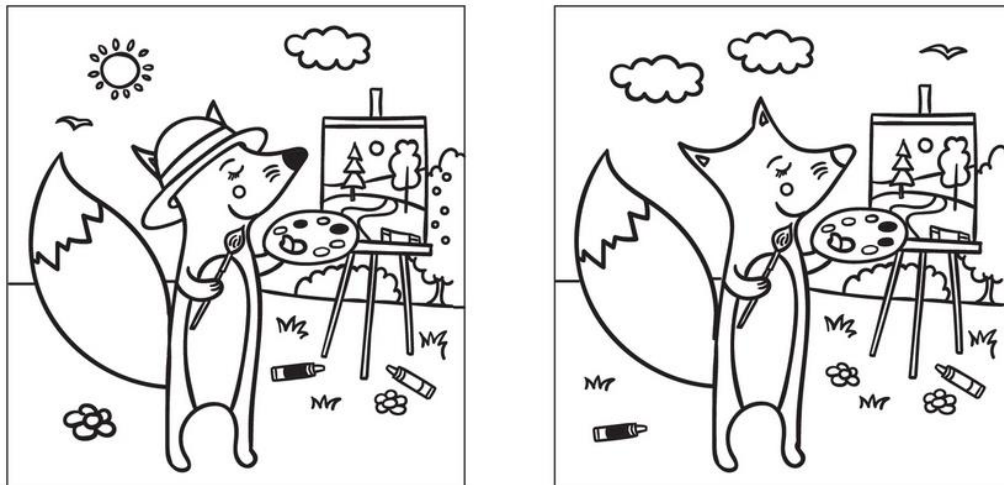
```
for (int i=1; i ≤ s.length(); i++)
    for (int j=1; j ≤ t.length(); j++)
        if(s.charAt(i-1) == t.charAt(j-1))
            a[i][j] = min(a[i-1][j-1], a[i][j-1]+1, a[i-1][j]+1)
        else
            a[i][j] = min(a[i-1][j-1]+1, a[i][j-1]+1, a[i-1][j]+1)
```

\\ substitute cost = 1

```
return a[s.length()][t.length()]
```

Images

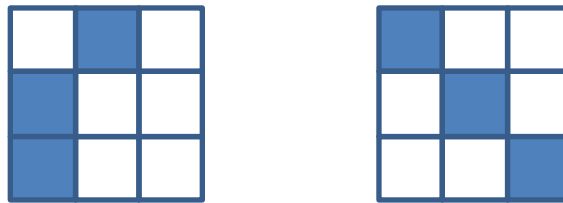
- How do you measure distances between images?



- Vector distance won't work
 - Fail when images are move by one pixel row
- Earth mover's distance

Earth mover's distance

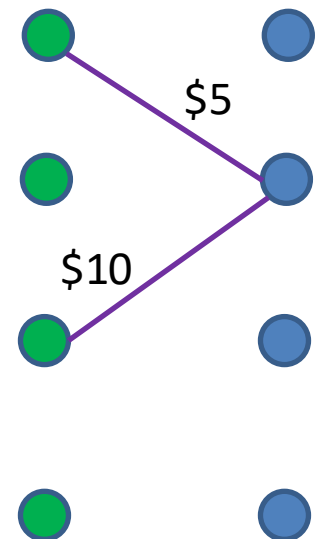
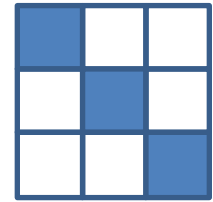
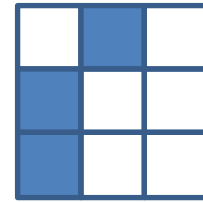
- Cost of moving pixels from one image to form the other



- Need to define the cost of vertical, horizontal, and diagonal movement.
- This is an example of an assignment problem

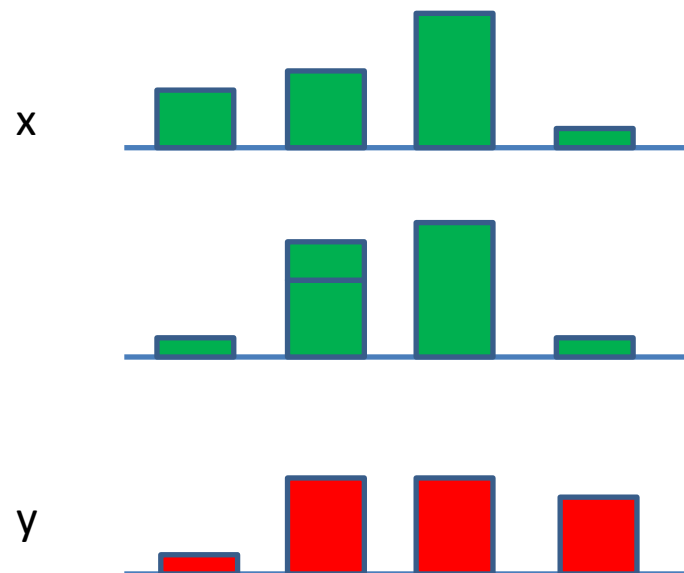
Earth mover's distance

- Assignment problem:
- Input:
 - N **workers**, N **tasks**
 - N*N **Matrix**: Cost of each worker doing each task
- Output:
 - 1-to-1 assignment of one worker to one task
 - Minimize total cost
- Min sum total matching
 - Solution: Hungarian algorithm
 - https://en.wikipedia.org/wiki/Hungarian_algorithm



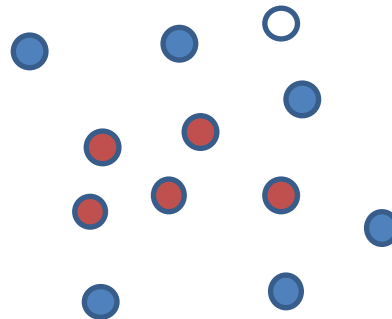
Earth mover's distance

- Earth mover's distance on vectors (1d)
 - Compute distance from $x = (3,4,7,1)$ to $y = (1,5,5,4)$
- $x = (3,4,7,1) \xrightarrow{2} (1,6,7,1) \xrightarrow{1} (1,5,8,1) \xrightarrow{3} (1,5,5,4) = y$
- Total cost: $2+1+3=6$



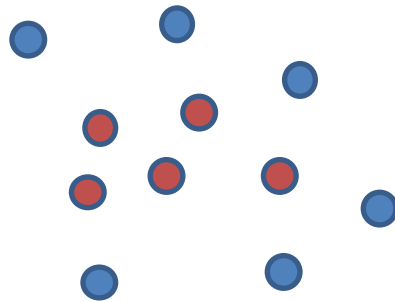
Nearest neighbor classifier

- How can we learn in spaces that aren't Euclidean?
- Take a **base** set
 - for example the entire sample
- For each new point, give it the same label as its nearest neighbor in the base set.



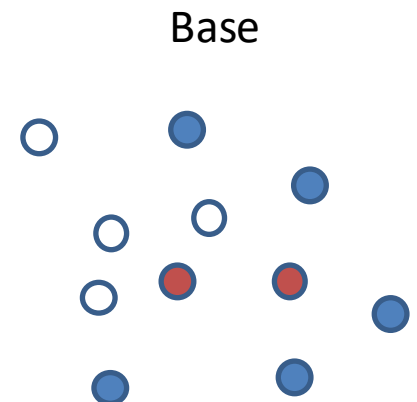
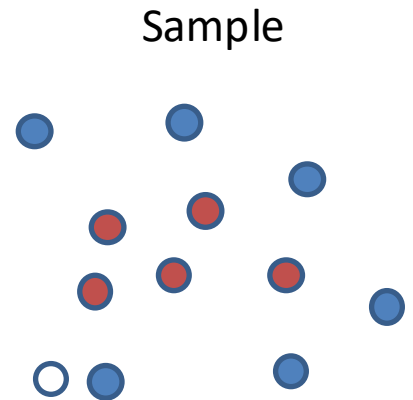
Nearest neighbor classifier

- This has infinite VC-dimension!
- Proof:
 - For every point set, for every possible labeling
 - take a base set having those points with that labeling.



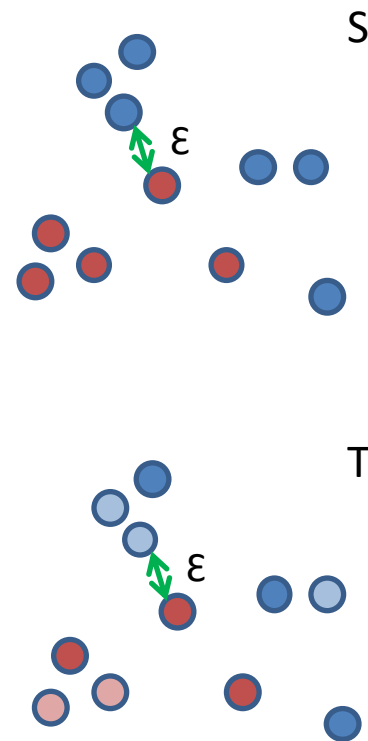
Generalization bounds

- k-NN
 - Rule: Take k nearest neighbors and majority vote
 - Good for de-noising.
- Condensing
 - Base set cardinality bounded by some s
 - Rule: NN in the base set
 - Example: sample S of size n , retain subsample $S' \subset S$ of size s as base set
 - On sample of size n :
 - Number of labelings: $O(n^s)$
 - VC-dimension: $O(s)$



Nearest neighbor condensing

- Given a training set S , Let ϵ be the *margin*
 - the minimum distance from red to blue points.
- An **ϵ -net** of S is a subset T satisfying the following:
 - **Packing**: for every p, q in T , $d(p, q) \geq \epsilon$
 - **Covering**: for every p in S , $d(p, T) < \epsilon$
 - Greedy construction:
 - Initialize $T = \emptyset$
 - For all $p \in S$, if $d(p, T) > \epsilon$, then add p to T .
- Take T as base for nearest neighbor.
 - 1-NN consistent on S – no empirical error
 - How big is T ? What's the VC-dimension?



Nearest neighbor condensing

- How big is T?
 - Ball of radius 1 contains all points
 - Points of T are at distance greater than $\epsilon < 1$
 - Balls of radius $\frac{\epsilon}{2}$ around points of T do not intersect
- Volume of d-dimensional ball of radius R
 - $V_d(R) = \frac{\pi^{\frac{d}{2}}}{\Gamma(\frac{d}{2}+1)} R^d$
 - $\Gamma(k)$ is Euler's Gamma function,
equal to $(k-1)!$ for k a positive integer
- Size of T $< \frac{V_d(1)}{V_d(\epsilon/2)} = (2/\epsilon)^d$

